Algorithmie

SAÉ 3.02 - Labyrinthe

Développement efficace

Étudiants :

Victor Bredelle
Antonin Marouze
Romain Harlaut
Angèl Zheng
Baptiste Lavogiez

Enseignant :

Mme Everaere

17 novembre 2025

Table des matières

1 Introduction **3**

2 Investissement dans le projet **4**

3 Structures de données pour les labyrinthes **6**
3.1 Labyrinthes parfaits 6
3.2 Labyrinthes aléatoires 6

4 Algorithmes de génération **7**
4.1 Distance entrée / sortie 7
4.1.1 Algorithme utilisé 7
4.1.2 Pseudo-code 8
4.1.3 Structures de données utilisées 9
4.1.4 Pertinence 10
4.2 Labyrinthes parfaits : Version TP 10
4.2.1 Algorithme utilisé 10
4.2.2 Pseudo-code 11
4.2.3 Structures de données utilisées 11
4.2.4 Pertinence 11
4.3 Labyrinthes parfaits : Version originale - Fusion 12
4.3.1 Algorithme utilisé 12
4.3.2 Pseudo-code 13
4.3.3 Structures de données utilisées 13
4.3.4 Pertinence 13
4.4 Labyrinthes aléatoires 14
4.4.1 Algorithme utilisé 14
4.4.2 Pseudo-code 14
4.4.3 Structures de données utilisées 15
4.4.4 Pertinence 15

5 Efficacité **16**
5.1 Méthode 16
5.1.1 Test de performance 16
5.1.2 Modélisation 17
5.2 Résultats 18

5.2.1 Labyrinthes aléatoires 18
5.2.2 Labyrinthes parfaits 18
5.2.3 Comparaisons inter-algorithmes 22
5.3 Conclusion 22

6 Complément et Bilan **23**
6.1 Distance entrée / sortie 23
6.2 Placement des entités 23
6.3 Déplacement des monstres 23

7 Conclusion **24**

1 Introduction

Ce rapport présente la partie algorithmique du projet **SAÉ S3.02 - Labyrinthe**, une application permettant la génération de labyrinthes variés. L'objectif est de décrire notre solution et de démontrer sa pertinence, guidée par l'efficacité d'exécution et les exigences du sujet.

2 Investissement dans le projet

Groupe G3.3

- Victor Bredelle
- Antonin Marouze
- Romain Harlaut
- Angèl Zheng
- Baptiste Lavogiez

Nom	Victor	Antonin	Romain	Angèl	Baptiste	Total
Part	0	0	0	0	0	100%

Table 2.1 – Répartition de l'investissement

1. Victor

2. Antonin

3. Romain

4. Angèl

Algorithme de déplacement de monstre.

5. Baptiste

- Algorithme distance entrée / sortie.
- Algorithme pour les labyrinthes aléatoires.
- Benchmarks de performance des labyrinthes et modélisation.

Le groupe

Chaque section est écrite par la personne l'ayant réalisée, et éventuellement relue par le groupe.

3 Structures de données pour les labyrinthes

3.1 Labyrinthes parfaits

Les labyrinthes parfaits peuvent être représentés à l'aide de structures de données simples : * Un tableau de booléens représentant les murs verticaux * Un tableau de booléens représentant les murs horizontaux

L'objectif est de conserver uniquement les informations essentielles dans la structure de données la plus simple possible, composée uniquement de variables primitives, tout en minimisant au maximum la redondance.

3.2 Labyrinthes aléatoires

Puisque les labyrinthes aléatoires héritent des labyrinthes parfaits dans notre cas, leur structure est identique.

4 Algorithmes de génération

4.1 Distance entrée / sortie

Cet algorithme sera utilisé dans les algorithmes à venir.

Le placement d'une distance entrée / sortie sera ici considéré comme un algorithme à part. Le respect strict de la distance nécessite en effet de réaliser un parcours intelligent vers la sortie, depuis l'entrée, la distance les séparant.

4.1.1 Algorithme utilisé

Une approche par niveaux sera utilisée ici. L'algorithme sera un **parcours en largeur**.

4.1.2 Pseudo-code

On note :

- d : la distance cible entre l'entrée et la sortie.

```
créer une position aléatoire entrée dans le labyrinthe

créer une file vide
créer une map distances vide
créer une liste positionsResultat vide
ajouter entrée à la file
ajouter (entrée, 0) à distances
niveauActuel = 0
distanceReelle = 0

tant que la file n'est pas vide :
    tailleNiveau = taille de la file

    si niveauActuel > d :
        arrêter le parcours

    si niveauActuel = d :
        pour i de 0 à tailleNiveau :
            position = retirer premier élément de la file
            ajouter position à positionsResultat
            distanceReelle = niveauActuel
            arrêter le parcours

    pour i de 0 à tailleNiveau :
    positionCourante = retirer premier élément de la file
    pour chaque direction {haut, bas, gauche, droite} :
        prochaine = calculer position adjacente selon direction
        si prochaine valide et prochaine non visitée et pas de mur :
            ajouter (prochaine, niveauActuel + 1) à distances
            ajouter prochaine à la file
```

```
31
32      distanceReelle = niveauActuel
33      niveauActuel = niveauActuel + 1
34
35  si positionsResultat est vide :
36      pour chaque (position, distance) dans distances :
37      si distance = distanceReelle :
38      ajouter position à positionsResultat
39
40  sortie = choisir aléatoirement une position dans
       positionsResultat
41  définir entrée dans le labyrinthe
42  définir sortie dans le labyrinthe
43  mettre à jour la distance effective avec distanceReelle
```

Notre solution permet alors un strict respect de la distance entrée / sortie, et surtout, un calcul de la distance maximale possible afin de ramener la distance voulue à celle maximale si celle voulue était impossible dans ce labyrinthe. Cette notion est **importante pour la suite**.

La mise à jour de la distance effective permet, entre autres, de prévenir la structure de labyrinthes que la distance voulue a changé, permettant un retour utilisateur.

4.1.3 Structures de données utilisées

Pendant l'exécution, l'algorithme utilise principalement une file et un dictionnaire.

- La file est indispensable puisqu'il s'agit d'un parcours en largeur. Elle nous permet d'explorer plusieurs niveaux à la fois (logique plus efficace).
- Le dictionnaire sert à associer une position à sa distance. Elle permet

Quant au retour, l'algorithme utilise un objet "distanceResultat" contenant :

- Une liste de positions.
- La distance effective.

Puis, il choisit aléatoirement une position parmi la liste. Sera attribuée au labyrinthe la distance effective, afin d'avertir de l'impossibilité de celle initialement voulue.

4.1.4 Pertinence

La motivation principale quant au choix des structures est l'efficacité. La structure choisie dans ce cas est minimale et comporte uniquement ce dont nous avons besoin pour déterminer une position aléatoire respectant la distance demandée (ou maximum si impossible).

4.2 Labyrinthes parfaits : Version TP

4.2.1 Algorithme utilisé

L'algorithme utilisé est celui du parcours en profondeur. L'objectif est de placer des murs partout dans le labyrinthe, puis de le parcourir en supprimant les murs gênants au passage. Cela garantit que le résultat sera un labyrinthe parfait.

4.2.2 Pseudo-code

```
1. Créer une position aléatoire comme entrée dans le
   labyrinthe
2. Créer une pile vide de positions
3. Créer un tableau de booléens "visitée" initialisé à
   false
4. Ajouter l'entrée à la pile
5. Marquer l'entrée comme visitée

6. Tant que la pile n'est pas vide :
    a. oldPosition = sommet de la pile (peek)
    b. directions = [haut, bas, gauche, droite]
    c. Mélanger l'ordre des directions aléatoirement
    d. positionValideTrouvée = false

    e. Pour chaque direction dans directions :
        i. position = oldPosition + direction
        ii. Si position est dans le labyrinthe ET pas
           encore visitée :
            - Marquer position comme visitée
            - Ajouter position à la pile
            - Supprimer le mur entre oldPosition et
              position
            - positionValideTrouvée = true
            - Break (sortir de la boucle des directions)

    f. Si positionValideTrouvée == false :
        - Retirer oldPosition de la pile (pop)
```

4.2.3 Structures de données utilisées

Pendant l'exécution, l'algorithme utilise principalement une pile et un tableau de booléens.

4.2.4 Pertinence

La pile est essentielle pour le parcours en profondeur, permettant de revenir en arrière lorsque nécessaire. Le tableau de booléens permet de suivre les cellules déjà visitées, évitant ainsi les cycles et garantissant que chaque cellule est visitée une seule fois.

4.3 Labyrinthes parfaits : Version originale - Fusion

4.3.1 Algorithme utilisé

L'algorithme utilisé est celui de la fusion aléatoire. Son objectif est d'attribuer un numéro différent à chaque case. Tant que toutes les cases n'ont pas le même numéro, on choisit deux cases adjacentes et on fusionne leurs numéros. Cela génère un labyrinthe parfait, mais avec une apparence différente, généralement jugée meilleure de celle obtenue par le parcours en profondeur.

4.3.2 Pseudo-code

```
génère une grille de numéro différents

1. Générer une grille où chaque cellule a un numéro unique
2. Tant que la grille contient au moins deux numéros diffé
   rents :
    a. Choisir une position aléatoire (pos)
    b. Choisir une direction aléatoire (dir)
    c. positionVoisine = pos + dir

    d. Si positionVoisine est dans le labyrinthe ET que les
        numéros de pos et positionVoisine sont différents :
        i. Supprimer le mur entre pos et positionVoisine
        ii. Fusionner les cellules : toutes les cases ayant
            le numéro de positionVoisine prennent le numéro
            de pos
```

4.3.3 Structures de données utilisées

Pendant l'exécution, l'algorithme utilise principalement une matrice d'entiers pour les numéros des cellules.

4.3.4 Pertinence

La matrice d'entiers est essentielle pour suivre les numéros des cellules, permettant de déterminer rapidement si deux cellules peuvent être fusionnées. Cette structure est simple et efficace pour l'objectif de l'algorithme.

4.4 Labyrinthes aléatoires

L'algorithme de génération de labyrinthe aléatoire héritant de l'algorithme parfait, cette sous-section est détaillée après la précédente.

L'objectif sera de partir d'un labyrinthe parfait et de le rendre imparfait. Cette solution permet un respect des propriétés "parfaites" par héritage.

Le chemin entre l'entrée et la sortie est, par définition, garanti, puisqu'un labyrinthe parfait est un graphe connexe.

La propriété connexe demeure inchangée tant qu'aucun mur n'est ajouté.

Cet algorithme n'ajoute aucun mur, il ne fait qu'en enlever.

Puisqu'un labyrinthe aléatoire est ici un graphe connexe, il existe toujours un chemin entre l'entrée et la sortie.

4.4.1 Algorithme utilisé

L'algorithme consiste à, pour un pourcentage de murs noté p, parcourir les murs, et, en enlever par un tirage de probabilité (1 - p).

Concernant la distance entrée / sortie, puisque sans paramètre dans le sujet, elle sera ramenée au maximum possible du labyrinthe, comme prévu par l'algorithme de placement.

4.4.2 Pseudo-code

On note :

- m : un labyrinthe parfait
- p : le pourcentage de murs souhaité

```
pour y allant de 0 à hauteur - 1
  pour x allant de 0 à largeur
    si mur horizontal à [y][x]
      si valeur aléatoire entre (0,1) inférieure à (1 - p)
        enlever mur horizontal à [y][x]
pour y allant de 0 à largeur - 1
  pour x allant de 0 à hauteur
    si mur vertical à [y][x]
      si valeur aléatoire entre (0,1) inférieure à (1 - p)
        enlever mur vertical à [y][x]
```

4.4.3 Structures de données utilisées

Les structures utilisées sont identiques à celle de l'algorithme parfait utlisé, que ce soit celui de *TP ou Fusion*.

4.4.4 Pertinence

La pertinence est donc identique.

5 Efficacité

5.1 Méthode

La présente sous-section expose notre méthodologie de manière théorique, divisée en deux parties distinctes.

1. Test de performance
2. Modélisation

5.1.1 Test de performance

On note :

- a : l'algorithme du labyrinthe,
- n : la taille initiale du labyrinthe,
- e : l'écart entre deux tests successifs,
- f : la taille maximale (fin du test).

$$\forall n > 2,\ n \in \mathbb{N}, \forall e \in \mathbb{N} :$$

```
créer un fichier csv de nom "a_n_e_f"
tant que n < f :
    t1 = mesurer le temps actuel
    générer un labyrinthe d'algorithme (a) de taille n (
       largeur = n, hauteur = 2*n, distance = n)
    t2 = mesurer le temps actuel
    écrire sur la ligne actuelle (n, t2 - t1)
    n = n + e
    passer à la ligne suivante
```

Information

Lorsque la distance entrée est de 0, l'algorithme de distance entrée/sortie ne s'exécute pas. Cela permet d'observer le coût brut de la génération ou de la distance entrée/sortie, plutôt qu'une limitation à une combinaison des deux. Cette idée sera appliquée dans certaines analyses.

Taille (n)	Temps de génération (ms)
10	12
15	18
20	25
25	30
30	38
35	45
⋮	⋮
495	1200
500	1250

Table 5.1 – Exemple de sortie pour $a = \text{Quelconque}, n = 10, e = 5, f = 500$

Implémentation Java

5.1.2 Modélisation

Ces fichiers sont modélisés, seul ou à plusieurs, par un script (python, matplotlib).

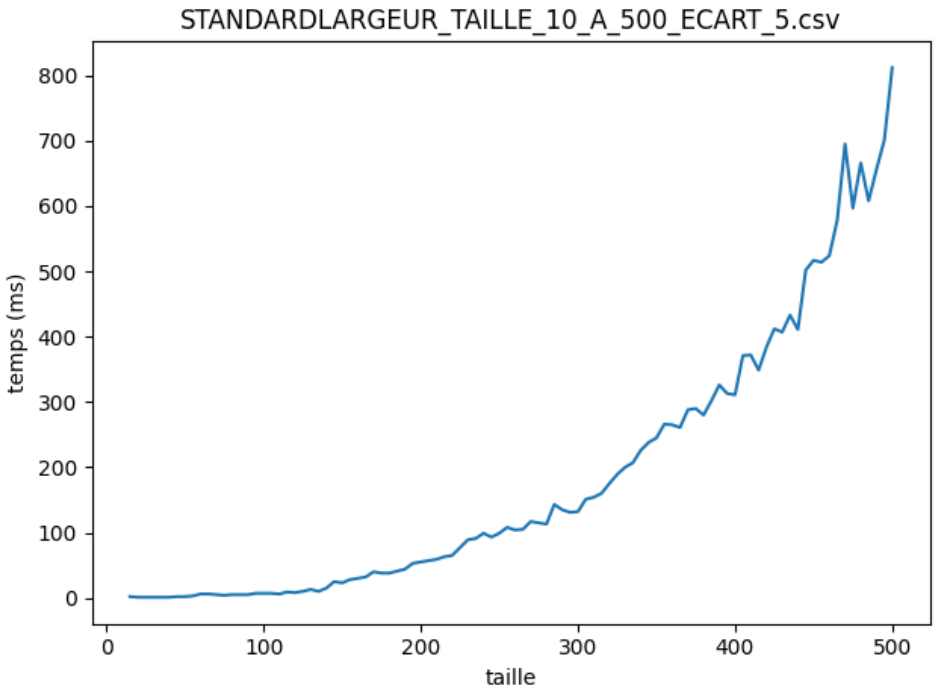

Figure 5.1 – Exemple de sortie pour $a =$ Quelconque, $n = 10, e = 5, f = 500$

Seront uniquement présentés en résultat ces tables et modélisations graphiques.

5.2 Résultats

5.2.1 Labyrinthes aléatoires

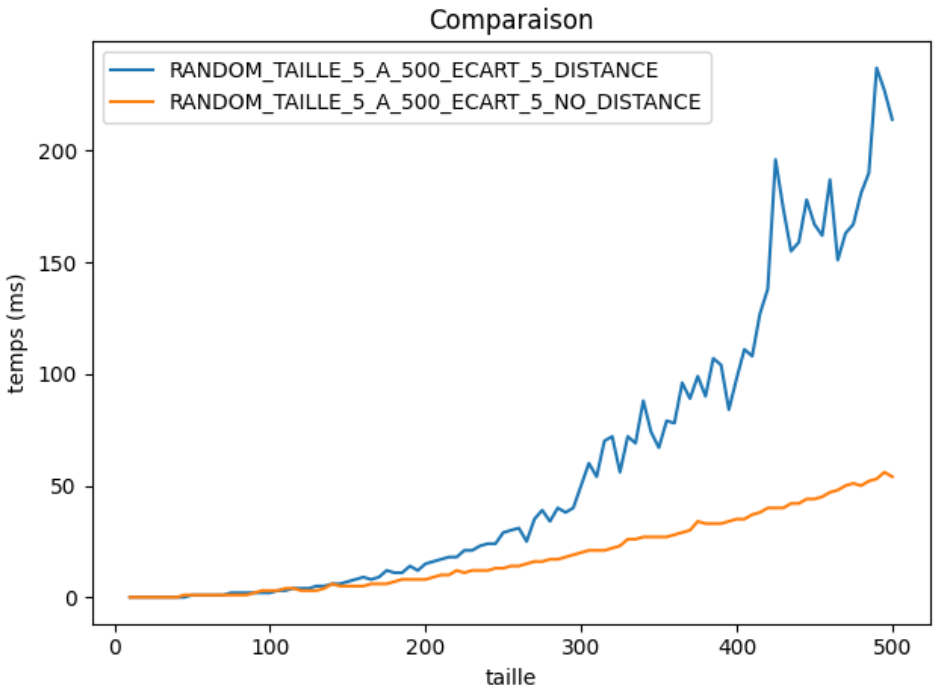

Figure 5.2 – $a = \text{RANDOM}, n = 5, e = 5, f = 500$

Taille	Temps (ms)
10	0
50	1
100	2
150	7
200	15
250	29
300	50
350	67
400	98
450	167
500	214

(a) RANDOM — Distance

Taille	Temps (ms)
10	0
50	1
100	3
150	5
200	8
250	13
300	20
350	27
400	35
450	44
500	54

(b) RANDOM — Sans distance

Figure 5.3 – Résultats RANDOM pour les tailles 10–500.

Observation

La distance entrée a un impact crucial puisqu'elle est maximale dans le cadre des labyrinthes aléatoires (spécification du sujet). Ainsi, l'algorithme devra trouver la position la plus éloignée, par le seul moyen de tout explorer, le maximum étant inconnu à chaque génération.

Ce maximum peut toutefois s'estimer : pour n=500, soit un labyrinthe de taille 500x1000, elle **est concentrée entre 10 000 et 20 000**.

5.2.2 Labyrinthes parfaits

Algorithme vu en TP

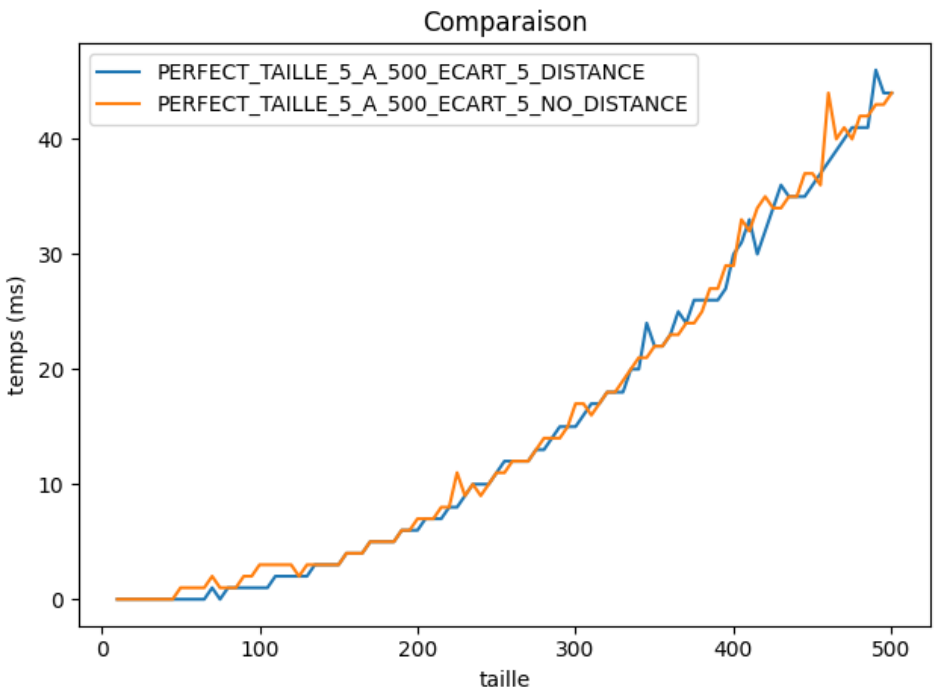

Figure 5.4 – $a = \text{PERFECT}, n = 5, e = 5, f = 500$

Taille	Temps (ms)
10	0
50	0
100	1
150	3
200	6
250	11
300	15
350	22
400	30
450	36
500	44

(a) PERFECT — Distance

Taille	Temps (ms)
10	0
50	1
100	3
150	3
200	7
250	11
300	17
350	22
400	29
450	37
500	44

(b) PERFECT — Sans distance

Figure 5.5 – Résultats PERFECT pour les tailles 10–500.

Observation

La distance entrée a un impact très mince puisqu'elle n'est pas maximale (les niveaux ne sont ainsi pas tous explorés).

Algorithme original : Fusion

Observation

La complexité explose lorsque n dépasse 100, problème induit par la nature de l'algorithme (génération aléatoire par grille).

5.2.3 Comparaisons inter-algorithmes

Parfaits / Fusion

Observation

La métrique étant proche lorsque n $<$ 100, une approche équilibrée consisterait à utiliser, dans une implémentation, l'algorithme de fusion pour de petits labyrinthes plus harmonieux, et l'algorithme de profondeur pour une exécution instantanée.

Parfaits / Aléatoires

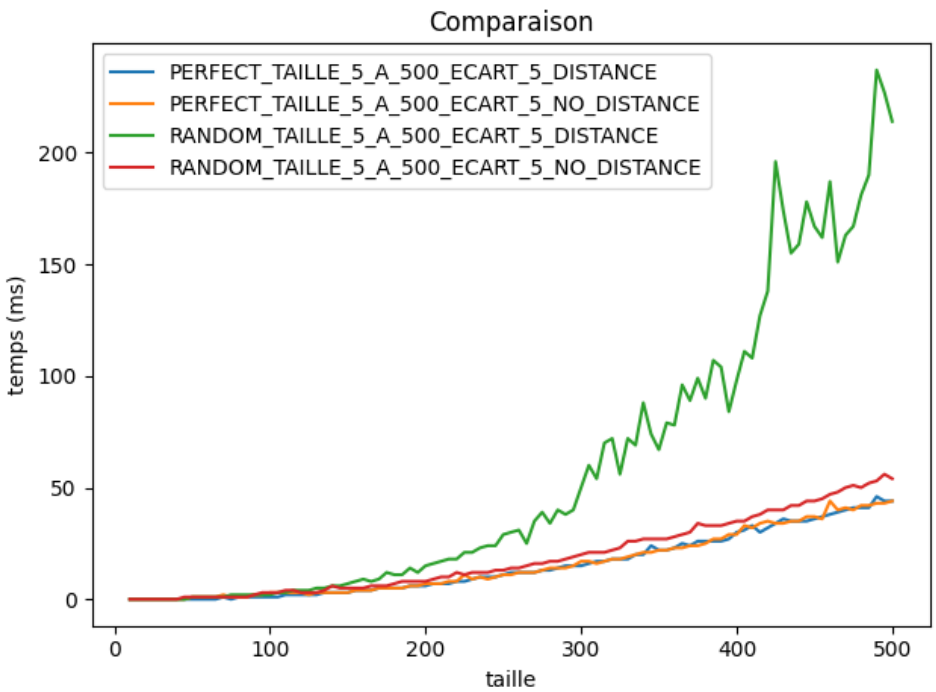

Figure 5.6 – Comparaison PERFECT (TP) / RANDOM

Observation

Comme prévu, les labyrinthes ont une efficacité très proche, la seule différence notable étant conceptuelle (distance entrée / sortie imposée)

5.3 Conclusion

6 Complément et Bilan

6.1 Distance entrée / sortie

Notre solution actuelle est optimisée, mais elle n'est apparue qu'après plusieurs tentatives. En effet, notre première solution consistait à :

- Faire un dictionnaire (clé : Position, valeur : Distance) de toutes les positions depuis la position en paramètre par parcours en largeur.
- Puis, choisir toutes les positions satisfaisant

En réalité, cette approche est inefficace puisqu'elle implique de parcourir absolument tout le labyrinthe. Elle avait été imaginée pour provisionner une solution simple au problème du maximum de la distance entrée / sortie.

Les performances de l'algorithme parfait ont été multipliées par 10 après application de la nouvelle solution.

6.2 Placement des entités

Notre projet comporte des **entités** (*joueurs, monstres, sorties...*) qui doivent être placées initialement selon des stratégies. Le problème étant très similaire à celui du placement de l'entrée et de la sortie, un algorithme de la sorte a été implémenté pour le calcul de leurs positions de départ.

La différence étant que le placement vérifie l'absence d'entités sur cette case. L'algorithme enlève alors les positions occupées pour le niveau exploré. Si le niveau demandé ne comporte aucune position valide, alors nous explorons un niveau supérieur.

6.3 Déplacement des monstres

Notre projet comporte des **monstres** qui sont des entités qui cherchera à toucher le joueur mettant ainsi fin à sa partie.

Le monstre se déplacera en fonction du meilleur chemin vers le joueur, donner par une fonction implémentant un parcours en largeur, le monstre choisira donc toujours le chemin le plus optimal vers le joueur.

7 Conclusion